Kestrel Flight Software: MVP Spec

Oct 27, 2026 · @Matthew

Goal and scope

The Kestrel MVP is flight software for a simulated CubeSat, written in C on FreeRTOS's POSIX port so it runs on a laptop. It takes commands from a ground station, sends telemetry, and falls back to SAFE mode on any fault. It shows four things flight software teams screen for: RTOS task design, a binary packet protocol, a mode and fault state machine, and requirements traced to tests.

In scope:

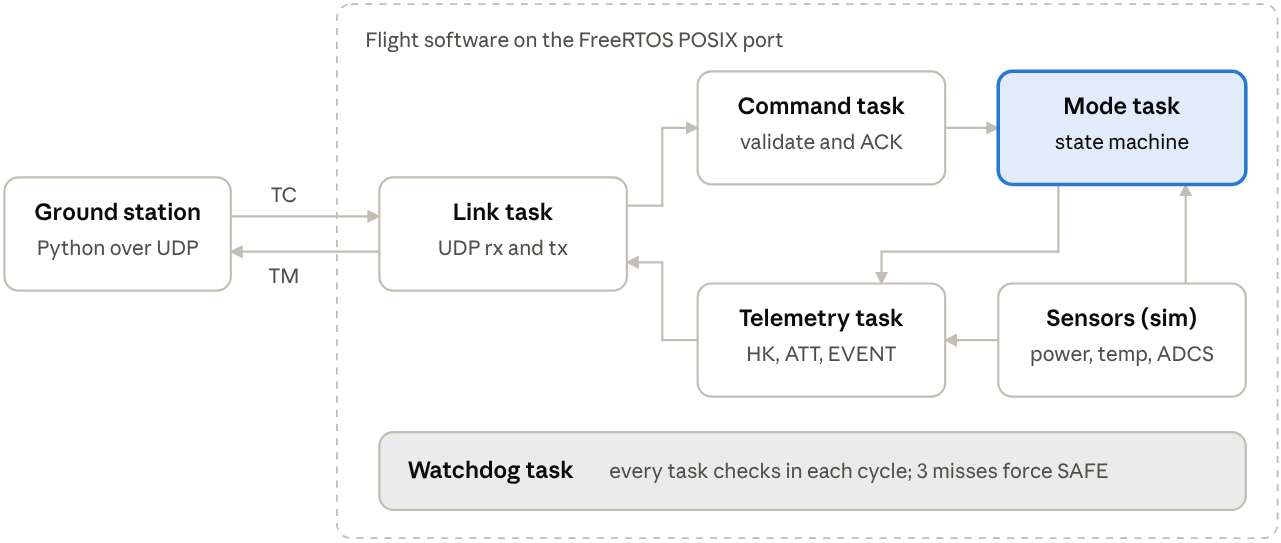
- 6 FreeRTOS tasks in C11, communicating only through queues
- CCSDS space packets: 4 telemetry types and 6 commands, each with a CRC-16
- 4 modes: BOOT, SAFE, NOMINAL, LOW_POWER
- Simulated power, temperature, and attitude sensors, with fault injection
- A Python ground station talking to the flight software over local UDP
- 20 numbered requirements, each verified by a named test or inspection

Out of scope for the MVP: real hardware, a real radio link, attitude control, file transfer, encryption, and time-tagged commands. Hardware comes in phase 2.

Architecture

Six FreeRTOS tasks, each owning one job, pass data only through queues. That keeps every task testable on its own and makes a stalled task easy to detect.

Tasks talk only through queues, and any stall forces SAFE mode



Kestrel architecture · 6 tasks, queues between them

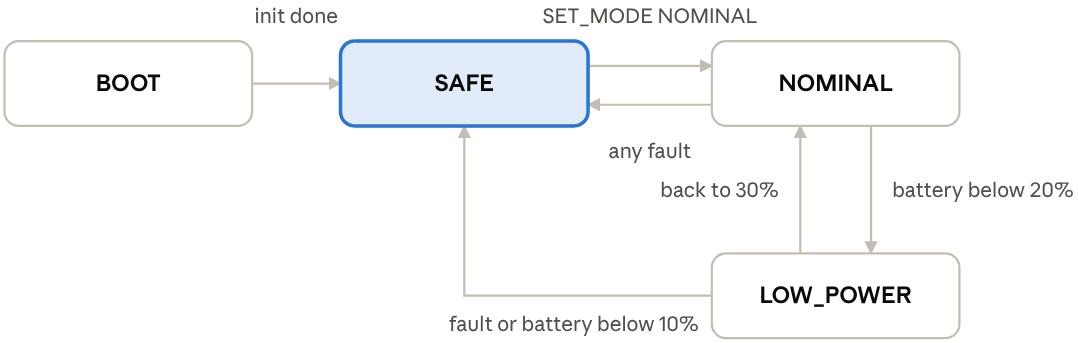
Commands (TC) come in through the link task and reach the command task, which validates them and asks the mode task to act. Sensor readings and mode events flow to the telemetry task, which sends packets (TM) back out through the link task.

Task	Priority	Runs	Owns
Watchdog	Highest	Every 1 s	Task check-ins, stall detection
Mode	High	On each event	Current mode, transition rules
Command	High	On each command	Validation, ACKs, command counters
Link	Medium	On each packet	UDP socket in and out
Telemetry	Medium	At HK and ATT rates	Packet building, sequence counts
Sensors	Low	Every 100 ms	Simulated battery, temperature, rates, fault injection

Operating modes

The software boots into SAFE and only goes to NOMINAL when the ground tells it to. Low battery drops it to LOW_POWER and returns it automatically. Every other fault lands in SAFE.

Every fault path ends in SAFE, and only a ground command leaves it



SET_MODE SAFE from the ground works in any mode.

Operating modes · 4 states, 6 transitions

The 20% and 30% thresholds give hysteresis, so a battery hovering near 20% does not flip modes every second.

Mode	HK telemetry	ATT telemetry	Commands accepted
BOOT	None	Off	None
SAFE	Every 10 s	Off	NOOP, SET_MODE, REQUEST_HK, RESET_COUNTERS
NOMINAL	1 Hz, adjustable	1 Hz	All
LOW_POWER	Every 5 s	Off	All

Packet format

Every packet is a CCSDS space packet: a 6-byte primary header, a 4-byte secondary header with mission elapsed time in milliseconds, the data field, then a CRC-16/CCITT over everything before it. All fields are big-endian.

Primary header field	Bits	Kestrel value
Version	3	0
Type	1	0 = telemetry, 1 = command
Secondary header flag	1	1
APID	11	See the table below
Sequence flags	2	3 (unsegmented)
Sequence count	14	Per-APID counter, wraps after 16383
Data length	16	Bytes after the primary header, minus 1

APID	Name	Direction	Contents	Rate
0x010	HK	Telemetry	Mode, uptime, battery mV and %, temperature, commands accepted and rejected, last fault code	1 Hz in NOMINAL
0x011	ATT	Telemetry	Simulated angular rates x, y, z	1 Hz, NOMINAL only
0x012	EVENT	Telemetry	Event ID, severity, two parameters	On each event
0x013	ACK	Telemetry	Echoed command sequence count, status code	Per command
0x100	CMD	Command	Function code byte, then arguments	From ground

Code	Command	Arguments
0x00	NOOP	None
0x01	SET_MODE	Target mode

Code	Command	Arguments
0x02	RESET_COUNTERS	None
0x03	SET_HK_RATE	Period in ms, 250 to 10000
0x04	REQUEST_HK	None, sends one HK packet now
0x05	INJECT_FAULT	Fault ID (test builds only)

Fault management

Any fault the software detects sends it to SAFE, and only a ground command brings it back to NOMINAL. Each fault also emits an EVENT packet.

Fault	Detection	Response
Task stall	A task misses 3 watchdog check-ins in a row (checked every 1 s)	SAFE
Low battery	Battery below 20%	LOW_POWER
Critical battery	Battery below 10%	SAFE
Over or under temperature	Temperature outside -20 to 60 C	SAFE
Bad command	Bad CRC, unknown APID or code, wrong length, argument out of range	Rejected, error ACK, rejected counter +1

Fault injection: INJECT_FAULT can stall a named task, drain the battery, push the temperature out of range, or drop a sensor. It is compiled in only when the build sets `KESTREL_TEST`, so flight builds cannot trigger it.

Requirements

The MVP is done when all 20 requirements pass verification. Test means an automated test whose name contains the ID, such as `test_fsw_012_watchdog_safe`.

ID	Requirement	Verified by
FSW-001	The software shall finish initialization and enter SAFE within 2 s of start.	Test
FSW-002	The software shall leave SAFE only on a SET_MODE command.	Test
FSW-003	The software shall send HK telemetry at 1 Hz, within 50 ms, in NOMINAL.	Test
FSW-004	The software shall accept HK periods from 250 to 10000 ms and reject others.	Test
FSW-005	Every packet shall carry a valid CCSDS primary header and CRC-16.	Test
FSW-006	Each APID's sequence count shall increase by 1 per packet and wrap after 16383.	Test
FSW-007	The software shall reject commands with a bad CRC, unknown APID or code, or wrong length.	Test
FSW-008	The software shall send an ACK with a status code within 100 ms of each command.	Test
FSW-009	HK shall report counts of accepted and rejected commands.	Test
FSW-010	The software shall enter LOW_POWER below 20% battery and return to NOMINAL at 30% or above.	Test
FSW-011	The software shall enter SAFE within 1 s of battery below 10% or temperature outside -20 to 60 C.	Test
FSW-012	The software shall enter SAFE within 4 s when a task misses 3 watchdog check-ins.	Test
FSW-013	In SAFE, the software shall accept only NOOP, SET_MODE, REQUEST_HK, and RESET_COUNTERS.	Test
FSW-014	The software shall send an EVENT packet for every mode change and fault.	Test

ID	Requirement	Verified by
FSW-015	The software shall send ATT telemetry only in NOMINAL.	Test
FSW-016	The software shall not allocate memory after initialization.	Inspection
FSW-017	Tasks shall share data only through FreeRTOS queues or mutex-protected state.	Inspection
FSW-018	Fault injection shall be compiled only in test builds.	Inspection
FSW-019	The flight code shall build with -Wall -Wextra -Werror and pass cppcheck with no findings.	Inspection
FSW-020	The ground station shall decode every telemetry type and send every command.	Demo

Ground station

The ground station is one Python 3 script that sends commands and decodes telemetry over local UDP. The flight software listens for commands on 127.0.0.1:5000 and sends telemetry to 127.0.0.1:5001.

- `ground/ccsds.py`: packs and parses headers, checks CRCs. The integration tests reuse it.
- `ground/gs.py`: a command prompt (`mode nominal`, `hk-rate 500`, `inject stall link`), a live one-line HK readout, and an event log.
- Every telemetry packet is also written to a CSV file, so a run can be replayed or plotted later.

The C and Python packet code must produce identical bytes. A shared set of known-good test vectors checks this from both sides.

Testing and traceability

CI fails if any requirement has no passing test, so the traceability is enforced, not just written down.

- **Unit tests (C, Unity framework):** CRC, header packing, sequence wrap, the command validator, and the mode state machine written as a pure function from (mode, event) to the next mode.
- **Integration tests (pytest):** start the flight binary, send commands over UDP, and check telemetry content and timing. There is at least one test per requirement marked Test.
- **Trace check (`tools/trace.py`):** reads the requirements table, finds each ID in test names or inspection notes, prints the matrix, and fails if any ID is missing.
- **CI (GitHub Actions, Ubuntu):** build with warnings as errors, cppcheck, unit tests, integration tests, then the trace check.

Repo layout

All network code sits behind a small link layer in `hal/`, so phase 2 can swap UDP for a UART without touching the tasks.

```
kestrel/
  fsw/
    src/
      main.c
      tasks/      link.c  cmd.c  mode.c  tlm.c  sensors.c  wdog.c
      ccsds/      packet.c  crc16.c
      hal/        udp_link.c   (phase 2: uart_link.c)
    include/
    FreeRTOSConfig.h
  ground/         gs.py  ccsds.py
  tests/
    unit/         (Unity, C)
    integration/  (pytest)
    vectors/      known-good packets shared by C and Python
  tools/          trace.py
  docs/           requirements.md  icd.md  (packet definitions)
  CMakeLists.txt
  .github/workflows/ci.yml
  README.md
```

Milestones

Six milestones, each ending in something that runs. Only claim a milestone on your resume after it is finished.

- [ ] **M0 Skeleton:** the FreeRTOS POSIX demo builds with CMake, two tasks run, and CI is green.
- [ ] **M1 Packets:** CCSDS packing and CRC in C and Python, with shared test vectors and unit tests.
- [ ] **M2 Link:** the link task is up, the ground station shows HK at 1 Hz, and a NOOP gets an ACK.
- [ ] **M3 Modes:** the mode task and state machine, SET_MODE, EVENT packets, and SAFE's command rules.
- [ ] **M4 Faults:** simulated sensors, the watchdog, and fault injection, with FSW-010 to FSW-014 passing.
- [ ] **M5 Ship:** the requirements doc, the trace check in CI, and a README with the architecture diagram and a short demo GIF. Make the repo public.

Definition of done: all 20 requirements verified, CI green, and a demo that injects a fault, shows the drop to SAFE, and recovers it with a ground command.

Phase 2 and resume targets

Phase 2 moves the same flight software onto a real board, which turns it into hardware-in-the-loop work.

- Port to an STM32 Nucleo board (F446RE or L476RG) running FreeRTOS.
- Replace `udp_link.c` with a UART link, so the ground station talks over USB serial.
- Replace the simulated attitude data with a real IMU over I2C (BNO055 or MPU-6050).
- Run the same integration tests against the board. Capture I2C and UART traces on a logic analyzer for the README.

Resume bullets to aim for. Fill in the real numbers once each part is built:

- Built flight software for a simulated CubeSat in C on FreeRTOS: 6 tasks, CCSDS telemetry and commands, and a mode state machine with watchdog-driven safe mode.
- Wrote 20 requirements with traceability enforced in CI, verified by N unit and integration tests, including fault injection for task stalls, power, and thermal faults.